

Hopfield–VAMP: Associative Addressing, Predictive-Coding Routers, and Temporal Attractors for Virtually Addressed Parameter Memories

Ben Goertzel, Claude Fable 5

(companion note to MacDonald’s “Virtually Addressed Memory for Parameters,” July 2026 draft)

July 9, 2026

EARLY DRAFT – consolidating a design conversation; errors and omissions abound!

Abstract

MacDonald’s Virtually Addressed Memory for Parameters (VAMP) reframes continual learning as resource-bounded management of an addressable, compressible graph of parameter states, but leaves the addressing mechanism abstract: the address $a(x)$ is described as “inferred or amortized,” without a committed mechanism. We argue that modern Hopfield networks supply that mechanism almost exactly, and that the correspondence is structural rather than merely analogical. First, we show that hetero-associative modern Hopfield retrieval over stored delta-atom keys *is* VAMP addressing, with the inverse temperature β acting as a continuous knob interpolating the paper’s limiting cases: $\beta \rightarrow 0$ recovers the SGD single-state limit, intermediate β yields metastable cluster states that identify consolidation candidates for free, and large β recovers exact leaf recall, preserving VAMP’s exact-recall proposition. Second, we make the addressing process itself a predictive-coding (PC) learner over the *address stream*, so that the same energy-based discipline governs both node-level modeling and routing; applying VAMP recursively to the router’s own parameters yields a natural notion of *second-order VAMP*. Third, we add a *temporal module* that supplies the one thing static content addressing structurally lacks – sequence – so that routing can resolve ambiguous addresses using task history and can compile reusable itineraries through the parameter DAG. We argue that this module is best implemented not as an asymmetric Hopfield network storing sequences as dynamical attractors, but as a small Hopfield-transformer over the address stream augmented with an episodic store of past routing trajectories: since attention is itself modern Hopfield retrieval, temporal structure is carried in context rather than in weights, the itinerary store becomes append-only like the value store, and the sequence router therefore does not forget. The asymmetric dynamical version is retained as an illuminating alternative instantiation. Finally, we assemble these pieces into a two-store cognitive architecture in which the symmetric net owns the values and the temporal module owns only the itinerary, arbitrated by an entropy-gated mixing coefficient and bootstrapped in stages as consolidation builds the macro-node alphabet that the temporal module sequences. The architecture makes a sharp falsifiable prediction: the prototype routing episodes distilled by the temporal store and the macro-node paths compiled by parameter-graph consolidation should identify the same structure – one discovered from routing history, the other from compression pressure. We extend MacDonald’s seed research plan with experiments that test this.

1 Introduction

The VAMP framework [MacDonald, 2026] proposes that a continually learning neural system should maintain not a single mutable parameter vector but an addressable directed acyclic graph of parameter states, deltas, and compressed macro-nodes. The effective parameters used for a datum x are an expression over this graph,

$$\theta_{\text{eff}}(x) = \theta_0 + \sum_j a_j(x) \Delta_j, \quad (1)$$

where the address $a(x)$ is obtained by solving (or amortizing) an addressed inference problem

$$(z^*, a^*) = \arg \min_{z, a} [E(x, z, \theta_G(a)) + \lambda_{\text{addr}} R_G(a)]. \quad (2)$$

VAMP is deliberately noncommittal about *how* a is inferred. The paper notes that $a(x)$ is “not merely a learned feed-forward router,” and lists content indices, latent keys, and error signatures as address-proposal mechanisms, but the addressing computation itself is a placeholder.

This note argues that the placeholder should be filled by associative memory in the modern Hopfield sense [Ramsauer et al., 2021, Krotov and Hopfield, 2016, Demircigil et al., 2017], and develops the consequences in four stages of increasing commitment:

1. **Symmetric Hopfield addressing** (§3): retrieval over delta-atom keys is a descent step on a log-sum-exp energy [Ramsauer et al., 2021], so address inference and latent inference become the same kind of computation, and the inverse temperature β becomes a single continuous control for the exact-recall $\leftrightarrow$ consolidated-generalization tradeoff (the extra step – from an accurate address through θ_{eff} to the downstream energy – is bounded in Proposition 1).
2. **Predictive coding on the addressing net** (§4): the router is trained by PC on the address stream – predict the next useful address given current state and residual error – so that routing learns by the same local discipline as the nodes it routes among.
3. **Second-order VAMP** (§5): the router’s own parameters are themselves managed as a (small) VAMP graph, closing the recursion and eliminating the embarrassment of a continual-learning architecture whose addressing mechanism forgets catastrophically.
4. **Temporal module** (§6): a mechanism supplying the one thing static content addressing lacks – sequence – so that routing can disambiguate on task history and compile reusable trajectories through the parameter DAG. We develop two instantiations and recommend the second: an asymmetric Hopfield network storing sequences as dynamical attractors (§6.2), and – the default – a Hopfield-transformer over the address stream with an episodic trajectory store (§6.3), which keeps the temporal store append-only and forgetting-free.

Section 7 assembles these into a two-store architecture with an explicit arbitration rule and a staged bootstrap; §8 maps the result onto Goertzel’s columnar residual architecture [Goertzel, 2026] and reads column–template fusion as a bidirectional associative memory; §9 catalogs failure modes; §10 extends MacDonald’s seed research plan. Because attention is modern Hopfield retrieval [Ramsauer et al., 2021], the entire architecture – value store, router, and temporal module – is built from a single associative-memory primitive at three scales, a uniformity we return to in the conclusion.

A terminological aside that is more than a pun: “virtually addressed memory” is practically a synonym for Kanerva’s Sparse Distributed Memory [Kanerva, 1988], and attention – hence modern Hopfield retrieval – has been shown to closely approximate SDM [Bricken and Pehlevan, 2021]. The Hopfield instantiation therefore quietly closes a loop back to the framework’s own name.

2 Background

2.1 VAMP in brief

We adopt the notation of MacDonald [2026]. A parameter-memory graph $G_t = (V_t, E_t)$ contains a root θ_0 , frozen delta atoms Δ_j , and expression codes c_v with $\theta_G(c_v) = \theta_0 + \sum_j c_{v,j} \Delta_j$. Online training performs addressed inference (2), forms a sparse low-description-length delta, stores a new node or edge, and runs bounded background graph maintenance (consolidation). The exact-recall proposition states that if an address c_i with $\theta_G(c_i) = \theta_i^+$ remains reachable, minimum energy on x_i never degrades. SGD is the one-node, immediate-lossy-consolidation limit; mixture-of-experts (MoE) is the shallow, architecture-fixed routing limit.

2.2 Modern Hopfield networks in brief

A modern (dense) Hopfield network over stored patterns $K = [k_1, \dots, k_M] \in \mathbb{R}^{d \times M}$ has energy

$$E_{\text{Hop}}(q) = -\beta^{-1} \log \sum_{j=1}^M \exp(\beta k_j^\top q) + \frac{1}{2} \|q\|^2 + \text{const}, \quad (3)$$

and the one-step update

$$q' = K \text{softmax}(\beta K^\top q) \quad (4)$$

is a convergent descent step on (3) [Ramsauer et al., 2021]. Storage capacity is exponential in d [Demircigil et al., 2017], retrieval typically converges in one update, and the hetero-associative variant with separate keys K and values V , $\xi' = V \text{softmax}(\beta K^\top q)$, is exactly the attention mechanism. Crucially for what follows, Ramsauer et al. [2021] characterize the fixed-point structure as a function of β and pattern separation: well-separated patterns at high β are retrieved individually; clusters of similar patterns at intermediate β produce *metastable* fixed points near cluster means; and as $\beta \rightarrow 0$ the unique fixed point is the global mean of all stored patterns.

3 Symmetric Hopfield Addressing

3.1 The mapping

VAMP’s Algorithm 1 already records, per stored atom, a content key, latent state, error signature, achieved energy, and usage statistics. Collect these into a key $k_j \in \mathbb{R}^d$ per delta atom Δ_j (or per macro-node), and let $q(x) \in \mathbb{R}^d$ be a query computed from the current datum – e.g. an embedding of x , the base model’s residual error signature on x , or a concatenation. Define the address by hetero-associative Hopfield retrieval:

$$a(x) = \text{softmax}(\beta K^\top q(x)), \quad \theta_{\text{eff}}(x) = \theta_0 + \sum_j a_j(x) \Delta_j. \quad (5)$$

This is not an approximation of VAMP addressing; it *is* an instance of it, with three properties the abstract version lacks:

1. **Energy semantics.** Retrieval (5) is a descent step on (3). The addressed inference problem becomes a nested energy minimization,

$$\min_{z, q} E(x, z, \theta_G(\text{softmax}(\beta K^\top q))) + \lambda_{\text{addr}} [\eta T_{\text{iter}} + H[a]], \quad (6)$$

where T_{iter} counts Hopfield update iterations and $H[a]$ is the entropy of the retrieval distribution. Both terms make $\lambda_{\text{addr}} R_G(a)$ concrete: iterations are literal compute spent on recall, and entropy penalizes diffuse, expensive-to-interpret addresses. Address inference and latent inference are now the same kind of computation, which keeps the whole construction inside the predictive-coding / energy-based framing rather than bolting a foreign router onto it.

2. **One-shot, forgetting-free writes.** Writing a new atom to the memory is appending a column to K (and Δ storage). No gradient training of the router, hence no interference in the routing function when new atoms arrive. A learned MoE-style router is itself a continually trained network subject to catastrophic forgetting; it is awkward for a continual-learning architecture to harbor forgetting inside its own addressing mechanism. Append-plus-occasional-consolidation is also much closer to the hippocampal fast-store / neocortical slow-store picture of complementary learning systems [McClelland et al., 1995].
3. **Iterative refinement as slow recall.** VAMP’s Algorithm 2 step 5 says: if the fast solution is uncertain or high-energy, spend additional addressing cost on slower recall. Here that is literal: run more Hopfield iterations, or lower-level retrievals within the region selected by a coarse retrieval (see §3.4).

3.2 β is the fast/slow and exact/consolidated knob

Mapping the Ramsauer fixed-point regimes onto VAMP’s spectrum of limiting cases gives the central observation of this section:

Hopfield regime	Fixed-point structure	VAMP interpretation
$\beta \rightarrow 0$	global mean of stored patterns	SGD limit: one collapsed state
intermediate β	metastable cluster means	macro-nodes / consolidated regions
large β , separated	sharp single-pattern recall	exact leaf recall

Two consequences follow.

Proposition 1 (Exact recall survives in the high- β regime, under a basin-preserving encoder). *Suppose after training on x_i the system stored key k_i and delta expression such that $\theta_G(e_i) = \theta_i^+$, where e_i is the one-hot address selecting that expression. Assume (i) k_i remains stored with minimum separation $\Delta_{\text{sep}} = \min_{j \neq i} \|k_i - k_j\|$ from other keys; and (ii) the query encoder is basin-preserving for x_i , i.e. $q(x_i)$ lies in the retrieval basin of k_i . Then for*

$$\beta \gtrsim \frac{1}{\Delta_{\text{sep}}} \log \frac{M}{\epsilon}, \quad (7)$$

where $M = |V|$ is the number of stored keys, one-step retrieval yields an address within ϵ of e_i in total variation, and hence

$$\min_{z, q} E(x_i, z, \theta_G(\text{softmax}(\beta K^\top q))) \leq \min_z E(x_i, z, \theta_i^+) + \tilde{\epsilon}(\beta, M),$$

with $\tilde{\epsilon} \rightarrow 0$ as the right-hand side of (7) is satisfied. VAMP’s exact-recall proposition is thus preserved, up to retrieval precision, by the Hopfield instantiation.

Proof sketch. The softmax leakage onto the $M-1$ competing keys is bounded by $(M-1) \exp(-\beta \Delta_{\text{sep}} \|q\|)$ up to the usual inner-product normalization; requiring this to fall below ϵ gives (7). Assumption (ii) is what places $q(x_i)$ where that bound applies – it is a property of the encoder, not of the store.

The composition step (address $\rightarrow \theta_{\text{eff}} \rightarrow$ downstream energy) is Lipschitz in the address under bounded $\|\Delta_j\|$, so an ϵ -accurate address gives an $O(\epsilon)$ energy gap. The full constant depends on the downstream energy’s smoothness and is left implicit. $\square$

Two consequences of (7) deserve emphasis, and both qualify the “single evaluation-time knob” framing.

Remark 1 (β is a schedule, not a constant, and the encoder carries the guarantee). *First, the required β grows like $\log M$: as a continual learner fills its store, fixed β at fixed separation lets retrieval precision decay, so the exact-recall regime migrates with lifetime and β must be scheduled against memory growth. The cost is only logarithmic, but the knob is coupled to M . Second, assumption (ii) relocates the hard part of the guarantee into the query encoder: exact recall holds only if the encoder keeps $q(x_i)$ in k_i ’s basin, which is precisely the “key drift / query mismatch” failure mode of §9. Proposition 1 should therefore be read as: the Hopfield store contributes exactness given a basin-preserving encoder; keeping the encoder basin-preserving is a separate obligation, discharged by freezing or slowly updating it and monitoring retrieval margin on a probe buffer.*

Remark 2 (Capacity under correlated keys). *The exponential-capacity results for modern Hopfield networks [Ramsauer et al., 2021, Demircigil et al., 2017] assume random or well-separated patterns. VAMP keys are structured task embeddings whose pairwise correlation grows over a lifetime as related regimes accumulate – exactly the regime where separation conditions are most likely to fail and Δ_{sep} shrinks, forcing β up faster than $\log M$ alone would suggest. Two mitigations keep this tractable: the locality fix (§3.3) confines dense mixing to within-region key sets that are deliberately kept small and separated, so the effective M and correlation in any one retrieval are bounded; and consolidation actively merges the most correlated keys, which is the same pressure that would otherwise degrade separation. Correlated-key capacity is nonetheless the most likely place for the associative-memory story to strain, and we flag it as the first thing an empirical study should stress-test.*

Claim 1 (Metastable states are consolidation candidates – with an adversarial caveat). *Patterns that fall into a shared metastable basin at intermediate β are precisely patterns whose keys (hence whose contexts of applicability) are similar. These are natural candidates for the consolidation operator: the Hopfield dynamics identifies mergeable regions as a side effect of retrieval, before any explicit graph-maintenance pass runs. Background consolidation can therefore be seeded by monitoring which stored keys co-inhabit metastable basins, replacing (or prioritizing) expensive replay-based merge search.*

This is candidate *generation*, not a merge decision, and the distinction is load-bearing. Metastable co-occupancy tracks key *similarity*, which the claim treats as a proxy for functional mergeability – but the motivating failure mode of §6.1 is exactly near-identical observations that require *incompatible* deltas. Those cases have near-identical keys, co-inhabit the same metastable basin, and are therefore flagged as merge candidates – the one merge that must not be performed. The adversarial false positives of basin-similarity sit *precisely* on the sequential-disambiguation cases the temporal module exists to handle. So we state as a hard rule:

No merge on similarity alone. A metastable co-occupancy is a proposal; a merge commits only after probe validation bounds the post-merge energy increase on retained examples, replay samples, or task probes.

Temporal veto on consolidation. The two stores can be made complementary *at the consolidation level*, not only at retrieval time. The episode store already records, per macro-node, the distribution of continuations that followed it. If two co-basin keys have *divergent continuation statistics* – routing through one is typically followed by different next-keys than routing through the other – that is direct evidence they are phase-distinguished and must not merge, even though their keys are similar. Formally, for candidate keys u, v flagged by co-occupancy, let P_u, P_v be their empirical next-macro-node distributions in the episode store and veto the merge when

$$D_{\text{JS}}(P_u \parallel P_v) > \tau_{\text{veto}}. \quad (8)$$

This turns the ambiguous-address hazard into a signal: the very cases where similarity-based consolidation is most dangerous are the cases where the temporal module has the evidence to stop it. It is a cleaner story than “monitor basins, then probe,” because it makes the temporal store an active participant in what the parameter graph is allowed to compress, and it closes the loop the other way from the vocabulary-coupling rule of §7 (there, consolidation feeds the temporal module; here, the temporal module constrains consolidation).

Lossy consolidation, in this picture, is literally temperature: lowering the effective β for a region blends its deltas; committing the blend and dropping the leaves is the lossy merge; retaining the leaves alongside the blend is lossless refactoring. The exact-recall $\leftrightarrow$ consolidated-generalization axis becomes a single scalar that can be swept *at evaluation time without retraining anything* – an experimental signature we exploit in §10.

3.3 The mixing caveat and the locality fix

The softmax in (5) produces a *mixture* of deltas, and mixing raw deltas is weight-space arithmetic. MacDonald’s own Git Re-Basin warning [Ainsworth et al., 2023] applies with full force: a metastable blend of incompatible deltas from different loss basins can be worse than either endpoint, since raw parameter combinations are not invariant to permutation symmetries or basin structure.

The fix is to make the Hopfield memories *local*. Maintain one associative memory per column or per macro-node, storing only that region’s atoms (e.g. the rank-one atoms A_i of a column C_a in the columnar architecture, §8). Soft mixing then only ever occurs among atoms created in the same basin, which are plausibly linearly composable – they were produced as small deltas from a shared parent, which is exactly the condition under which task-vector arithmetic is empirically benign. Across regions, retrieval should be sharp (high β , near-one-hot), i.e. selection rather than blending. Formally: a two-level address $a = (a^{\text{coarse}}, a^{\text{fine}})$ with $\beta_{\text{coarse}} \gg \beta_{\text{fine}}$.

3.4 Multilayer / hierarchical retrieval

Vanilla Hopfield retrieval is flat, while VAMP addresses are paths and expressions over a DAG. Compositional addressing is recovered by *iterated* retrieval: the query retrieves a parent macro-node; the retrieved state, concatenated with the residual error signature obtained after applying it, becomes the next query; that retrieves a child delta; and so on – a recurrent Hopfield walk down the graph. Stacking Hopfield layers in the manner popularized by “Hopfield networks is all you need” [Ramsauer et al., 2021] gives the amortized coarse-to-fine version: a top layer over macro-node keys for fast focus, lower layers over fine deltas within the selected region. This reproduces VAMP’s fast/slow recall hierarchy with an actual mechanism instead of prose. In §6 the walk itself becomes a learned object – either a dynamical one (§6.2) or, by preference, a retrieval over past walks (§6.3).

4 Predictive Coding on the Addressing Net

The retrieval map (5) is training-free, which is its virtue; but the *query construction* $q(x)$ and, in the iterated setting, the *transition* from one retrieval step to the next, benefit from learning. We propose that this learning be predictive coding on the *address stream*.

Concretely, let s_t denote the addressing state at step t of a retrieval walk (the current retrieved key or macro-node representation, together with the current residual error signature r_t of the model under the so-far-composed parameters). A transition network T_ψ predicts the next useful address:

$$\hat{k}_{t+1} = T_\psi(s_t), \quad \mathcal{L}_{\text{addr}}(\psi) = \|\hat{k}_{t+1} - k_{t+1}^*\|_{\Sigma^{-1}}^2, \quad (9)$$

where k_{t+1}^* is the key actually selected by (possibly slow, possibly exhaustive) energy minimization – i.e. the router is trained to *predict the outcome of its own slow deliberation*, exactly the amortization pattern of predictive coding, with a local, layerwise error signal rather than backprop through the whole retrieval trajectory. The prediction error $\|\hat{k}_{t+1} - k_{t+1}^*\|$ doubles as the addressing analogue of a PC residual: when it is persistently large, the router’s model of the address stream is inadequate, which is a signal either to spend more addressing compute (slow recall) or to fork (novelty).

Three details keep this from misbehaving. First, **the target must be generated with the temporal bias off** ($\gamma = 0$ in (14)). Otherwise slow deliberation consumes u_t and the router is trained to predict its own influenced output – self-confirming feedback that locks in early routing habits. Target-generating recall is therefore run as pure content addressing; the router learns to amortize *unbiased* deliberation, not to agree with itself. Second, **the training distribution is skewed toward hard cases**: slow recall runs only when fast retrieval is uncertain, so the (s_t, k_{t+1}^*) pairs concentrate where addressing is difficult. This is probably desirable – the router’s parameters are spent where they matter – but it should be noted, and easy cases sampled occasionally to avoid drift on the bulk of the distribution. Third, **the PC training buffer is subject to the same rewriting rule as episodes**: when macro-nodes merge, the stored (s_t, k_{t+1}^*) pairs referencing them go stale and must be rewritten (or dropped) exactly as episodes are (§7), or the router trains against a vocabulary the store no longer uses.

The aesthetic and practical point is uniformity: the same energy-based, locally-trained discipline governs node-level modeling (PC or causal coding inside each parameter node, in MacDonald’s §10 sense) *and* routing among nodes. There is one kind of learning in the system, applied at two levels.

5 Second-Order VAMP

Once the router has parameters ψ , it is itself a continually trained network, and the original problem recurs one level up: routing patterns learned for early task regimes can be overwritten by later ones. The framework suggests its own remedy. Apply VAMP to ψ :

$$\psi_{\text{eff}}(x) = \psi_0 + \sum_k b_k(x) \Delta\psi_k, \quad (10)$$

with a (much smaller) key store over routing contexts and the same append-consolidate discipline. We call this *second-order VAMP*: the addressing machinery for the parameter memory is itself a parameter memory. How load-bearing this is depends on the temporal instantiation chosen in §6: it is essential for the asymmetric dynamical router (whose weights hold the routing knowledge and can forget), and largely optional for the default episodic-transformer router (whose knowledge lives in an append-only episode store, leaving only a small prefix encoder to protect). We present it in full because the asymmetric case needs it and because the recursion is conceptually clarifying.

Three remarks keep this from spiraling:

1. **The recursion bottoms out for capacity reasons.** The router is small (it operates over d -dimensional keys and a near-discrete macro-node alphabet, §7), so its VAMP graph is tiny, and a third-order store would be managing a handful of parameters – at some level plain replay or even joint training is cheaper than addressing, and the recursion should be truncated there. Second order is likely the last level that pays rent.
2. **Second-order addresses are first-class context.** The choice of router variant $b(x)$ is itself informative about regime identity, and can be fed back as a feature of the first-order query – cheap, and it gives the system a crude representation of “which routing era am I in.”
3. **Consolidation at second order is policy distillation.** Merging router deltas is merging routing policies; the natural probe set is a buffer of past (s_t, k_{t+1}^*) pairs, i.e. replay in *address* space, which is far cheaper than replay in data space.

6 The Temporal Module

6.1 Motivation: routing is sometimes irreducibly sequential

Flat content retrieval is memoryless. MacDonald’s *ambiguous address* failure mode – the same observation requires different incompatible parameter states, and no context variable distinguishes them – is exactly the situation where the missing context is *temporal*: the correct delta depends on task phase, on what was retrieved recently, on position within a routine. A softmax over static keys provably cannot resolve this; something carrying information about the trajectory so far can. Temporal machinery earns its keep precisely and only where flat retrieval structurally fails, which gives a clean deployment test: *is the routing problem sequential or contextual? If not, stay with the symmetric store.*

Two things are wanted from a temporal module: *sequential disambiguation* (identical observations, different histories, different correct deltas) and *reusable itineraries* (compile a frequently useful ordered sequence of retrievals into a single addressable routine). There are two natural ways to supply them, differing in *where the temporal structure is stored*. It can live in *weights*, as the dynamical attractors of an asymmetric network (§6.2); or it can live in *context*, as attention over the trajectory prefix and over a store of past trajectories (§6.3). The second fits VAMP’s own append-only, forgetting-free philosophy far better, and is our recommended default; we present the first because its $W = S + A$ picture is conceptually clarifying and because autonomous, open-loop routine generation may matter in some future agentic setting.

6.2 Instantiation A (alternative): asymmetric dynamical attractors

Asymmetric Hopfield-style networks store *sequences* as dynamical attractors: classical constructions use delayed asymmetric couplings so that convergence to pattern μ destabilizes it in favor of pattern $\mu + 1$ [Sompolinsky and Kanter, 1986, Kleinfeld, 1986]. The attractor taxonomy reads as routing primitives: a **fixed point** is single-step retrieval of a leaf or macro-node (the symmetric case, recovered); a **limit cycle or sequential attractor** is a reusable multi-step routing subroutine (“in this context, apply this ordered sequence of deltas”), i.e. compositional skill-as-trajectory; and **trajectory-dependent selection** (identical observations, different histories, different branches) is exactly the resolution of the ambiguous-address failure mode. On this reading the temporal attractors of the module *are* the paths of the parameter DAG.

General asymmetric coupling has no Lyapunov function, so one loses the “addressing = energy minimization” story and risks unwanted cycles or chaos. The recoverable version decomposes the

coupling as

$$W = S + A, \quad S = \frac{1}{2}(W + W^\top), \quad A = \frac{1}{2}(W - W^\top), \quad (11)$$

a Helmholtz-like split in which S defines a dissipative gradient flow whose basins are the memory contents while A contributes a solenoidal, divergence-free circulation that connects basins into sequences without (to leading order) moving them. If $\|A\|$ is subcritical relative to basin curvature,

$$\|A\| < c \cdot \lambda_{\min}(\nabla^2 E_S|_{\text{basin}}), \quad (12)$$

the fixed points of S persist and A only stitches them into trajectories. Then $\|A\|$ relative to basin curvature is simultaneously the stability knob and the how-sequential-is-the-router-allowed-to-be knob, and a weak form of the exact-recall proposition survives because the resting attractors remain critical points of the dissipative part.

Remark 3 (Why this is not the default). *Three costs push against this instantiation. (i) Sequence storage in asymmetric nets is classically fussier and lower-capacity than static pattern storage [Sompolinsky and Kanter, 1986]; dense associative memories [Krotov and Hopfield, 2016] bridge toward the modern high-capacity regime, but the specific combination of modern high- β content storage with learned asymmetric sequential dynamics is an active area that should be preceded by a targeted literature sweep rather than taken from this note. (ii) The temporal structure lives in trained weights, so it reintroduces exactly the thing the symmetric store was praised for avoiding – forgetting in the router – which then has to be bought back with two-timescale training, local (non-BPTT) updates, and the second-order machinery of §5. (iii) The stack of attractor dynamics is not a single global energy descent. The next instantiation avoids all three.*

6.3 Instantiation B (default): a Hopfield-transformer over the address stream

Since attention *is* modern Hopfield retrieval [Ramsauer et al., 2021], the temporal module can be a small causal transformer over the address stream, storing temporal structure in *context* rather than in weights. Tokens are the macro-node keys retrieved so far along the current routing walk (optionally enriched with the residual-error signature r_t and the outcome of each step); the model’s output at position t is the bias u_t over the next macro-node key – exactly the signal the arbitration gate of §7 consumes, so nothing downstream changes. Temporal context now enters through attention over the trajectory prefix. The ambiguous-address case is still resolved: two identical observations with different histories present different prefixes and therefore attend differently. But there is no coupling matrix to keep subcritical, no Lyapunov anxiety, and no classical sequence-capacity fragility. Each attention layer retains per-retrieval Hopfield energy-descent semantics; we note honestly that the *stack* is not a single global Lyapunov flow the way the symmetric store is – the energy story here is per-layer, not architecture-wide.

Episodic trajectory retrieval. The move that makes this instantiation genuinely strong is to give the transformer a second associative store whose patterns are *entire past routing episodes*. When the system completes a task segment, append the address sequence $(k_1, \dots, k_T)$ with meta-data – outcome/return ρ_m , context signature, and the task-phase index at which each key was visited – to an episode store. At run time the current prefix is a query; retrieval over stored episode prefixes returns matching precedents, and their continuations supply the next-key bias, *weighted by outcome* so that failed precedents do not bias as confidently as successful ones:

$$u_t = \sum_m \alpha_m(\text{prefix}_{\leq t}) w(\rho_m) \text{cont}_m, \quad \alpha = \text{softmax}(\beta_{\text{ep}} Q_{\text{prefix}}^\top K_{\text{episodes}}), \quad (13)$$

where cont_m is the recorded continuation of episode m and $w(\rho_m)$ is an outcome/advantage weight (near zero for failed episodes). Without w , (13) retrieves failures as confidently as successes – a decision-transformer-style return conditioning is the minimal fix. Two further refinements matter. The prefix encoder Q_{prefix} should carry a *phase/position feature*, not just a bag of visited keys: the same subsequence recurring at different task phases should match differently, and a single order-insensitive embedding would collapse them. (At coarse macro-node granularity this may matter less than at leaf level, but it is cheap to include and expensive to discover missing.) This is sequence *retrieval* rather than sequence *dynamics*, and it inherits everything the symmetric value store was praised for: one-shot appends, dense-Hopfield capacity, and *zero forgetting in the stored content*. The only learned component left is the small prefix encoder, trained by the address-stream PC objective of §4; it is tiny, slowly updated, and – if one still wants the belt and suspenders – second-order-VAMP-protected. Because most structure now lives in appended episodes rather than trained weights, second-order VAMP shrinks from a load-bearing subsystem to an optional footnote.

Relation to episodic-memory and retrieval methods. This construction has close published relatives, and the contribution is the VAMP framing and the β -unification, not the episodic store per se. The episode store is a cousin of Neural Episodic Control and Model-Free Episodic Control [Pritzel et al., 2017, Blundell et al., 2016], which use append-only key–value stores with kernel retrieval for fast, forgetting-free learning; and the symmetric value store’s nearest published relative is arguably the Discrete Key–Value Bottleneck [Träuble et al., 2023], which selects frozen values by content-addressed keys specifically for continual learning. The coarse-to-fine hierarchical retrieval of §3.4 is essentially product-key memory [Lample et al., 2019]. Equation (13) with outcome metadata is a short step from retrieval-augmented decision transformers, and the “language of skills” reading below connects to the options framework [Sutton et al., 1999] and Soar-style chunking [Laird et al., 1987] alongside production compilation. What is new here is embedding these in VAMP’s addressed-parameter-memory and reading them through the single associative primitive.

The temporal module as a language of skills. Trained on the address stream, this module is literally a small language model whose vocabulary is the macro-node alphabet and whose corpus is the system’s own routing history, retrieval-augmented from the episode store: the system learns *a language of its own skills*. This gives “routing subroutine” a cleaner referent than “limit cycle” ever had. A subroutine is now a *consolidated prototype episode*: consolidation over the episode store extracts frequent shared subsequences and merges near-duplicate episodes into prototypes, which is skill compilation in nearly the classical production-compilation sense – a temporally extended macro over primitive retrievals, in the spirit of the options framework [Sutton et al., 1999] and Soar-style chunking [Laird et al., 1987]. Both the temporal store and the parameter graph therefore run a consolidation process, and they should be discovering the same routines – one from routing history, one from parameter-space compression. This sets up the money experiment (§10, E4), whose independence caveat we take care with there.

What is given up, honestly. Two capabilities are traded away for the robustness. First, *autonomy*: a dynamical attractor can run a routine open-loop, generating a sequence from its own dynamics, whereas the retrieval router is always query-driven and needs the loop closed through observations each step. For routing this is arguably a feature – one *wants* the itinerary re-grounded in the current residual error at every step rather than free-running – but it is a real difference, and the asymmetric instantiation remains the better choice if open-loop routine generation is ever required. Second, *unbounded history*: recurrence integrates arbitrarily far back, while a context

window is finite. Since routing context is task-phase-scale and tokens are coarse macro-node symbols, a modest window over a small alphabet is cheap and usually sufficient, and episode retrieval extends effective range well beyond the window (the relevant distant past is fetched by content rather than carried in state).

7 The Two-Store Architecture

We now assemble the pieces. The design principle that organizes everything:

The symmetric store owns the values; the temporal module owns only the itinerary.

The two are not peers doing similar jobs; they do categorically different things, and they are asymmetric in *stakes*. The symmetric store is ground truth: it holds the actual delta atoms and macro-nodes, its energy descent carries the recall guarantee (Proposition 1), its writes are one-shot and forgetting-free. The temporal module holds nothing that cannot be afforded to lose: it is a prior over trajectories through the symmetric store’s keys. It never emits a delta; it emits a *bias over which key the symmetric store retrieves next*. If it is wrong, degraded, or cold-started, the system falls back to flat content retrieval and loses sequencing, not correctness. That invariant is what keeps any imperfection in the itinerary machinery from ever touching the recall guarantee – and with the default episodic-transformer instantiation (§6.3), where the itinerary store is itself append-only, “degraded” mostly means “no matching precedent yet,” not “forgot a routine it once knew.”

7.1 Arbitration: propose and dispose

Per retrieval step:

1. Symmetric retrieval on the current query gives a content-based address distribution $a^{\text{cont}} = \text{softmax}(\beta K^\top q_t)$.
2. The temporal module produces a bias u_t over the same (macro-node) key set: in the default instantiation, by attention over the trajectory prefix and retrieval from the episode store, (13); in the alternative instantiation, from the asymmetric net’s state.
3. The two are combined additively in logit space, gated by a coefficient that must distinguish *ambiguity* from *novelty*:

$$a_t = \text{softmax}(\beta K^\top q_t + \gamma_t u_t), \quad \gamma_t = g(H[a_t^{\text{cont}}]) \cdot \sigma(m_t - m_0), \quad (14)$$

where $H[\cdot]$ is the entropy of the content-retrieval distribution, g is increasing, $m_t = \max_j(\beta K^\top q_t)_j$ is the top logit magnitude (a retrieval-*margin* proxy), and σ is a sigmoid with threshold m_0 .

The two-factor form is the important correction over a pure entropy gate. High entropy in a^{cont} has two causes with *opposite* correct responses. **Ambiguity**: several stored keys match well – flat-and-high logits – and temporal context should break the tie (γ up). **Novelty**: *no* stored key matches – flat-and-low logits – and the correct move is to fork a new leaf, in which case the itinerary module’s confident bias toward familiar continuations is actively harmful and must be suppressed ($\gamma \rightarrow 0$). Entropy alone cannot tell these apart; the margin factor $\sigma(m_t - m_0)$ does, opening the gate only when *something* is actually matched. When both entropy is high and margin is low, the system forks rather than sequences. The gate also yields a free interpretability probe: logging γ_t (and its two factors) over a stream shows exactly where the task needed temporal context versus where it hit genuine novelty.

7.2 What goes in which store

The stores hold different pattern sets; do not mirror contents.

- The **symmetric store** indexes *all* atoms and macro-nodes: large, in the exponential-capacity dense-associative regime, hierarchically organized as in §3.4 (sharp coarse selection over regions, softer fine mixing within a region, per §3.3).
- The **temporal module** works over *macro-node keys only*: the near-discrete alphabet of consolidated regions. Sequencing is a coarse-grained, region-to-region phenomenon, not a raw-parameter one. Its tokens are macro-node symbols and its episode store holds trajectories over them (tens of symbols, not thousands of deltas), so the transformer stays small and the effective sequence-capacity issue is dodged by construction; its alphabet is literally the output of the consolidation process.

That last clause is an architectural dependency worth stating as a design rule: **consolidation defines the temporal module’s vocabulary**. Every time graph maintenance compiles a new macro-node, a symbol is added to the temporal module’s token/key space; every time it merges two, their temporal roles are merged, and episodes referencing them are rewritten. The two update on the same events, from opposite sides.

7.3 Staged bootstrap

The two-part split introduces a circularity the single-store design did not have: the temporal module can only learn good itineraries once macro-nodes exist to sequence, but early in a stream everything is a fresh leaf, consolidation has not run, and the coarse alphabet is empty. The honest design is *staged activation*:

Stage A. Pure symmetric content addressing. Atoms accumulate; the β -sweep behavior of §3.2 should already be observable.

Stage B. Consolidation comes online (seeded by metastable-basin co-occupancy, Claim 1); macro-nodes form; the coarse alphabet stabilizes.

Stage C. The temporal module is switched on over the macro-node alphabet with the entropy-gated γ of (14). In the default instantiation this mostly means beginning to log and retrieve episodes and training the small prefix encoder by address-stream PC; the episode store needs no warm-up beyond having a stable alphabet to write over.

Wiring the temporal module in from step zero is the tempting mistake: it would train on noise and the architecture would be blamed. The staging mirrors both the bootstrap implicit in the columnar architecture (columns presuppose discovered reusable regions) and the developmental ordering of complementary learning systems – the fast episodic store runs first; the slow structured store comes online once there is regularity to structure [McClelland et al., 1995].

7.4 The whole loop

Putting it together, one online step of Hopfield–VAMP:

1. Observe x_t ; form query q_t (embedding + residual error signature). The first query’s error signature is computed under θ_0 (no address has been applied yet – a mild chicken-and-egg); for heavily drifted regimes that signature can be saturated and uninformative, in which case

the iterated retrieval of §3.4 handles it: a coarse first retrieval on the embedding alone supplies a provisional θ_{eff} , and the sharper error signature computed under it drives the next retrieval step.

2. Coarse retrieval over macro-node keys, biased by the temporal module via (14); fine retrieval within the selected region (§3.3); compose θ_{eff} .
3. Settle latents z by PC/EBM inference under θ_{eff} ; if energy or address-prediction error is high, iterate retrieval (slow recall) or fork a new leaf (novelty).
4. If learning: form a sparse delta, append its key/value to the symmetric store (one-shot write); extend the current routing prefix with the chosen macro-node key. On successful completion of a task segment, append the whole prefix to the temporal module’s episode store (one-shot write).
5. Background: consolidation merges metastable co-occupants into macro-nodes; vocabulary changes propagate to the temporal module (its tokens and episodes are rewritten); episode-store consolidation distills frequent subsequences into prototype routines; optional second-order VAMP protects the small prefix encoder.

7.5 Compute accounting

Since the entire VAMP premise is *resource-bounded* management, the per-operation costs should be stated, because they are what λ_{addr} trades against. Let M be the number of stored atoms/macro-nodes, M_c the number within an active region (with $M_c \ll M$ under locality, §3.3), N_{ep} the episode-store size, L the context-window length, and d the key dimension.

Operation	Cost	Notes
Coarse retrieval	$O(M_{\text{macro}} d)$	over macro-node keys only
Fine retrieval	$O(M_c d)$	within selected region
Iterated (slow) recall	$\times T_{\text{iter}}$	the λ_{addr} knob
Temporal bias (default)	$O(L d + N_{\text{ep}} d)$	prefix attention + episode retrieval
Symmetric append	$O(d)$	one-shot write
Episode append	$O(T)$	one-shot, per completed segment
Consolidation pass	$O(R d + C_{\text{validate}})$	region-local + probe validation
Episode distillation	$O(N_{\text{ep}} \log N_{\text{ep}})$	subsequence mining, amortized

The dominating growth terms are retrieval in M and episode retrieval in N_{ep} . Hierarchy (§3.4) reduces the first from $O(M)$ to $O(M_{\text{macro}} + M_c)$ but does not eliminate it; episode distillation bounds the second by replacing raw episodes with a smaller prototype set. The resource-bounded knob is explicit: λ_{addr} sets how many iterations T_{iter} of slow recall are affordable, and the consolidation schedule sets how aggressively M and N_{ep} are held down against accuracy. This is the same memory/compute/accuracy tradeoff MacDonald frames at the parameter-graph level, now priced per operation.

8 Relation to the Columnar Residual Architecture

The mapping of MacDonald [2026] §11 – columns C_a as macro-deltas, gates $g_a(x)$ as amortized address coefficients, rank-one atoms A_i as delta bases – extends naturally under the Hopfield

reading, and the two additions MacDonald identifies as “not forced” by minimal continual learning both acquire mechanisms:

1. **Column-local exact recall.** MacDonald observes that mutable columns break exact recall, and that each column needs a local archival or delta-DAG mechanism. That mechanism can simply *be* a column-local symmetric Hopfield memory over the column’s atoms: one-shot writes preserve old atom expressions; high- β retrieval within the column recovers them; intra-column consolidation is the local β /merge process of §3.2. This is also exactly the locality fix of §3.3, so the same design choice solves the mixing caveat and the column-recall gap at once.
2. **Column–template fusion as a BAM.** The fusion energy term $-\beta s^\top B t$ coupling column selections s and symbolic template selections t is, read as a network, a bidirectional associative memory [Kosko, 1988] coupling two content-addressable stores – one over continuous column keys, one over symbolic (Atomese/MORK) template keys. Retrieval ping-pongs: columns bias templates, templates bias columns, settling to a joint minimum. This is a pleasingly classical interpretation of the neural–symbolic bridge: the symbolic head is not an exotic attachment but the second plate of a BAM whose first plate is the parameter memory. It also suggests the symbolic store should share the retrieval formalism (modern Hopfield over template keys), so that the joint settling has a single energy.

The temporal module adds a third connection: routing subroutines (prototype episodes over macro-nodes, §6.3) are the procedural counterpart of *ordered* symbolic templates – plans – and the BAM coupling gives a route by which a symbolic plan retrieved by the template store can bias the temporal module’s next-key prediction, and vice versa. A retrieved prototype episode is close to an executable macro; a retrieved symbolic plan is its declarative sibling; the coupling lets each cue the other. We flag this as design space rather than design: it is the natural meeting point of Hopfield–VAMP and the MeTTa/MORK-side machinery, and the right formalization likely belongs in the columnar architecture’s own idiom.

9 Failure Modes and Risks

Beyond MacDonald’s structural failure modes, which all persist, the Hopfield instantiation adds its own:

Blend toxicity. Metastable mixing of deltas across basins is weight-space arithmetic and can be destructive [Ainsworth et al., 2023, Yadav et al., 2023]. Mitigation: locality (§3.3); sharp coarse retrieval; treat cross-region blends as consolidation *proposals* to be validated by probes, never as silent inference-time behavior.

Key drift / query mismatch. Keys are frozen at write time, but the query encoder and base representations may drift, silently degrading recall without deleting anything – forgetting by *mis-addressing*. Mitigation: freeze or slowly update the query encoder; store queries alongside keys and monitor retrieval margin on a probe buffer.

Entropy-gate gaming. The γ gate trusts the temporal module exactly when content retrieval is confused; a badly trained temporal module then does maximal damage in exactly the hard cases. Mitigation: cap γ until the module’s address-prediction error on held-out transitions clears a threshold; fall back to slow symmetric search when both stores are uncertain.

Novelty misrouted as ambiguity. High content-retrieval entropy has two causes with opposite correct responses: several keys matching well (ambiguity – route to the temporal module) versus no key matching (novelty – fork, and suppress the itinerary’s confident bias toward familiar continuations). A pure entropy gate sends both to the temporal module, so on novel data the router actively pulls toward stale routines. Mitigation: the two-signal gate of (14), conditioning on retrieval margin m_t as well as entropy, so flat-and-low-margin triggers a fork rather than a temporal override.

Context-window truncation. The default temporal module carries recent history in a finite context window, so disambiguating information older than the window is lost unless it was written to the episode store (§6.3). Mitigation: keep tokens coarse (macro-node symbols) so the window spans many task phases cheaply; rely on episode retrieval for long-range precedent. (Under the alternative asymmetric instantiation the corresponding risk is instead sequence-storage fragility, §6.2, Remark 3 – mitigated by keeping $\|A\|$ subcritical per (12) and lit-sweeping before committing to a mechanism.)

Episode-store growth. One-shot episode appends make the trajectory store grow unboundedly, and stale episodes referencing merged or deleted macro-nodes accumulate. Mitigation: episode consolidation – distill frequent subsequences into prototypes, drop dominated or superseded episodes, rewrite episodes when their macro-nodes merge; this is the address-space analogue of the parameter-graph consolidation and shares its compute budget.

Bootstrap mis-staging. Activating the temporal module before the macro-node alphabet stabilizes trains it on noise (§7.3); conversely, waiting too long forfeits sequential disambiguation during the period it may matter. Mitigation: gate Stage C on alphabet stability metrics (macro-node churn rate), not wall-clock.

Two learners, one blame signal. When performance drops, it may be the content store (bad atoms), the addressing (bad retrieval), or the itinerary (bad sequencing). The architecture is diagnosable – retrieval entropy, address-prediction error, and per-store ablations separate the cases – but only if those probes are built in from the start.

10 Experimental Plan

We extend MacDonald’s seed plan (his §13), slotting into and after his Stage 3 (amortized addressing) and Stage 5 (columnar graft). Throughout, streams are non-IID task sequences; metrics are his RBII-style set (online performance, mean forgetting, memory growth, addressing cost, recovery latency) plus retrieval entropy, address-prediction error, and γ traces.

E1: The β -sweep curves (load-bearing). Symmetric store only. Train on a non-IID stream with exact leaf retention; at evaluation, sweep β and plot exact-recall performance against consolidated-generalization performance *without retraining anything*. This splits into two predictions, because §3.3 warns that intermediate- β blends across basins can be worse than either endpoint.

E1a (global sweep). A single global β over all keys. Prediction: *non-monotonic*, with blend-toxicity dips where intermediate β mixes incompatible cross-basin deltas [Ainsworth et al., 2023]. A failed smooth curve here does *not* falsify addressing-as-energy-descent – it reconfirms Git Re-Basin – so E1a is a test of the toxicity structure, not of the core premise.

E1b (locality-respecting sweep). Sweep β_{fine} within columns/regions while holding β_{coarse} sharp (§3.3). Prediction: the *smooth* continuous traversal of the SGD-limit $\leftrightarrow$ macro-node $\leftrightarrow$ exact-recall axis. This is the load-bearing curve; if it does not appear, the addressing-as-associative-energy-descent premise fails and the later stages are moot.

The contrast between E1a and E1b turns the mixing caveat from a caveat into a second testable prediction. Both also locate where on the curve each column of a columnar architecture should sit. As M grows, re-run to confirm the $\beta \gtrsim \Delta_{\text{sep}}^{-1} \log M$ schedule of (7): the exact-recall knee should migrate with store size.

E2: Addressing bake-off. Same stream, addressing mechanisms over identical atom stores, now including two cheap baselines that the temporal module must beat to justify its parameters: (i) kNN content index; (ii) learned feed-forward router; (ii') plain GRU router; (iii) symmetric Hopfield with keys = (content key, error signature) and annealed or query-dependent β ; (iii') trivial kNN over (prefix $\rightarrow$ next-key) pairs; (iv) *the default temporal module* – symmetric content retrieval + Hopfield-transformer over the address stream with episodic trajectory retrieval, two-signal gate (14); (v) the alternative – symmetric content retrieval + PC-trained asymmetric transition. Include ambiguous-address episodes where disambiguation *requires* temporal context, and novelty episodes where the correct move is to fork. Predictions: (i), (iii) fail the ambiguous cases (memoryless); (ii), (ii') may pass but forget routing under distribution shift; (iv) passes *and* retains, with no forgetting in the itinerary store; (v) also passes and retains but is fussier and needs second-order protection to match (iv) (Remark 3). **The prefix-kNN baseline (iii') is the key control: if (iv) does not beat it, the transformer is not earning its parameters** and the episode store alone suffices. The direct (iv)-vs-(v) comparison is the empirical case for the default choice.

E3: Metastable-basin consolidation seeding, with temporal veto. Compare consolidation guided by metastable co-occupancy (Claim 1) against replay-search consolidation: merge quality (post-merge energy increase on retained probes) versus consolidation compute. Critically, include the adversarial cases – near-identical keys requiring incompatible deltas – and test whether the continuation-divergence veto (8) rejects exactly those merges. The metric of interest is false-merge rate on the ambiguous cases, with and without the veto; the veto should drive it down without suppressing legitimate merges (measured by generalization retained after correct merges).

E4: Prototype-episode / macro-node-path coincidence (the money experiment). Run the full two-store system through Stages A–C with the default temporal module. Extract (a) the *prototype episodes* distilled by episode-store consolidation and (b) the *macro-node paths* compiled by parameter-graph consolidation. Both are discrete sequences over the same alphabet, so overlap is measured directly (e.g. sequence-set edit similarity), without extracting attractors from a nonlinear system.

Independence is the crux, and the naive setup is confounded. The vocabulary-coupling rule of §7 rewrites episodes when macro-nodes merge, so prototype distillation runs over a stream that parameter-graph consolidation has *already re-tokenized* – some coincidence is then built in by construction, and a positive result would be closer to a consistency check on the rewriting rule than a discovery. Two designs restore genuine independence, and at least one must be used:

E4a (raw-stream distillation). Log the original *leaf-level* address trajectories alongside the rewritten ones, and run episode consolidation on the raw pre-rewrite streams. The two sides

then see different evidence: prototypes from raw routing history, paths from parameter-space compression.

E4b (held-out counterfactual). Compare parameter-graph consolidation against a counterfactual episode consolidation run on held-out data the parameter graph never saw, so neither side’s vocabulary was shaped by the other’s evidence.

Only under E4a or E4b does the prediction carry its advertised weight:

Prediction 1. *When the two consolidation processes are given genuinely independent evidence, the prototype routing episodes distilled by the temporal store and the frequently-traversed consolidated paths of the parameter graph nonetheless identify the same structure – one discovered from routing history, the other from parameter-space compression.*

Coincidence (under the independence-restoring design) is the headline result: strong evidence that “routing subroutine” and “consolidated macro-node” are one object seen from two sides – or a precise localization of where the story breaks. (The alternative asymmetric instantiation admits the same experiment in its original, harder form: extract limit cycles and compare.)

E5: Second-order ablation. Freeze vs. plainly-train vs. VAMP-protect the router parameters ψ across regime shifts; measure routing retention (address-prediction error on early-regime transitions) and downstream task retention.

E6: Columnar graft with column-local Hopfield stores. MacDonald’s Stage 5, with each column’s archival mechanism implemented as a column-local symmetric memory (§8); test whether column-local high- β retrieval restores exact recall that mutable columns lose.

11 Conclusion

VAMP poses continual learning as the management of an addressable history of parameter states and deliberately leaves the addressing mechanism open. Modern Hopfield networks close that opening with unusual precision: retrieval is energy descent, so addressing joins latent inference inside one energy-based discipline; writes are one-shot appends, so the addressing mechanism itself does not forget; the inverse temperature is a single evaluation-time knob spanning VAMP’s limiting cases from the SGD collapse through macro-node consolidation to exact leaf recall; and metastable structure identifies consolidation candidates as a side effect of retrieval. Predictive coding on the address stream makes routing learn by the same local rule as the nodes it routes among. The temporal module then adds the one thing content addressing structurally lacks – sequence. Here the design settles on storing that sequence in *context* rather than in weights: a Hopfield-transformer over the address stream, augmented with an append-only episodic store of past routing trajectories, keeps the itinerary machinery forgetting-free like the value store it sits beside, dodges the classical fragility of asymmetric sequence dynamics, and turns “routing subroutine” into a concrete, inspectable object – a consolidated prototype episode – rather than a limit cycle to be extracted from a nonlinear system. Because attention is modern Hopfield retrieval, the whole architecture then rests on a single primitive at three scales: value store, content router, and temporal episode store are all associative memories. The asymmetric dynamical version is kept on the page for its clarifying $W = S + A$ picture and for the one capability it alone offers – open-loop routine generation – but it is the alternative, not the default, and the framework’s own append-only, forgetting-free instincts are what point the other way.

The resulting two-store architecture – symmetric values, contextual itinerary, entropy-gated arbitration, staged bootstrap – is a fairly rich cognitive architecture assembled from unusually old parts made to cooperate: associative memory, retrieval-augmented sequence modeling, bidirectional coupling to a symbolic store, and complementary fast/slow learning. Whether the parts cohere is an empirical question with an unusually crisp signature: sweep β and the recall–generalization curve should appear; run the full loop and the temporal store’s prototype episodes should land on consolidation’s macro-node paths. Either result would be worth knowing.

References

- Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations*, 2023.
- Charles Blundell, Benigno Uria, Alexander Pritzel, Yazhe Li, Avraham Ruderman, et al. Model-free episodic control. *arXiv preprint arXiv:1606.04460*, 2016.
- Trenton Bricken and Cengiz Pehlevan. Attention approximates sparse distributed memory. In *Advances in Neural Information Processing Systems*, 2021.
- Mete Demircigil, Judith Heusel, Matthias Löwe, Sven Upgang, and Franck Vermet. On a model of associative memory with huge storage capacity. *Journal of Statistical Physics*, 168:288–299, 2017.
- Ben Goertzel. A staged research plan for predictive-coding residual learning with symbolic heads, on top of frozen large language models, May 2026. Unpublished research plan.
- Pentti Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.
- David Kleinfeld. Sequential state generation by model neural networks. *Proceedings of the National Academy of Sciences*, 83(24):9469–9473, 1986.
- Bart Kosko. Bidirectional associative memories. *IEEE Transactions on Systems, Man, and Cybernetics*, 18(1):49–60, 1988.
- Dmitry Krotov and John J. Hopfield. Dense associative memory for pattern recognition. In *Advances in Neural Information Processing Systems*, 2016.
- John E. Laird, Allen Newell, and Paul S. Rosenbloom. Soar: An architecture for general intelligence. *Artificial Intelligence*, 33(1):1–64, 1987.
- Guillaume Lample, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, 2019.
- Daniel MacDonald. Virtually addressed memory for parameters. Draft manuscript, July 2026. See also: Resource-bounded incremental induction for continual learning, to appear in *Proceedings of AGI-26*.
- James L. McClelland, Bruce L. McNaughton, and Randall C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.

- Alexander Pritzel, Benigno Uria, Sriram Srinivasan, Adrià Puigdomènech, Oriol Vinyals, et al. Neural episodic control. In *International Conference on Machine Learning*, 2017.
- Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, et al. Hopfield networks is all you need. In *International Conference on Learning Representations*, 2021.
- Haim Sompolinsky and Ido Kanter. Temporal association in asymmetric neural networks. *Physical Review Letters*, 57(22):2861–2864, 1986.
- Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.
- Frederik Träuble, Anirudh Goyal, Nasim Rahaman, Michael Mozer, Kenji Kawaguchi, et al. Discrete key-value bottleneck. In *International Conference on Machine Learning*, 2023.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*, 2023.